Testing approach — HitBox Collectibles operations console

How this console is tested, and why it is tested that way. The "why" matters more than the file list: most of the decisions below are responses to something specific about *this* application, and a suite that ignores them is busy rather than useful.

What makes this application awkward to test

Four properties shape everything else.

1. There is no offline mode. The README is blunt about it: "*There is no mock and no offline mode — every screen reads live data.*" The in-process backend mock was deliberately deleted once the console started talking to the real API. So an end-to-end test either talks to a live backend or intercepts at the network boundary. There is no third option, and pretending otherwise by re-introducing a mock would recreate the drift that mock was removed for.

2. Identity is external and human-gated. Clerk holds identity; this app holds a projection. The instance accepts **email codes, not passwords** as a first factor, so no automated sign-in can complete without a human reading an inbox. Any strategy that assumes a scripted login is wrong here before it starts.

3. The UI is derived from permissions, not from roles. The sidebar, every page guard and every action button come from the permission keys in `/authz/me`. Nothing maps a role name to a menu — that coupling is exactly what the backend removed. A test that asserts "an admin sees these eleven items" re-introduces the coupling in the test suite and will be wrong for every other role.

4. Absence and zero mean different things. The rule the whole console is built around:

> *A section the caller cannot see is **omitted** from the JSON — not empty, not zero, not null.*

`{ "revenue": 0 }` means there was no revenue. A missing `revenue` key means you may not see revenue. A test that only checks "a number rendered" cannot tell those apart, and neither can an operator if the UI gets it wrong.

The shape of the suite

```
e2e/
  support/
    routes.ts            the route manifest: path, <h1>, permission, nav group
    api-surface.ts       all 88 endpoints + the 9 operator workflows
    json.ts              narrowing helpers for unknown payloads (never `any`)
    session.ts           is there a captured session, and is it usable
    authed.ts            authenticated test base + open() / settled()
    a11y.ts              axe, scoped to WCAG A/AA
    capture-session.ts   the interactive login (npm run e2e:login)
  specs/
    public/              runs with no session — 214 tests
    authed/              runs with a captured session — 258 tests
playwright.config.ts
```

Two projects, split by what they need — not by what they cover

	NEEDS	RUNS WHEN	PURPOSE
`public-*`	a dev server	every commit, CI	The session wall, the whole API surface, both auth screens, a11y, responsive, hygiene
`authed-*`	a captured Clerk session	before a release, after touching the shell or authz	Every console screen, the API contract, permission gating, the nine workflows, resilience

The split is structural, and it is what keeps CI honest. The public project has no external prerequisite beyond `next dev`, so it can gate every merge. The authenticated project needs a human-captured session, so it cannot — and pretending it can is how suites end up permanently yellow and universally ignored.

Without a session the authenticated project does not fail; each test skips with an actionable reason. **A red suite that means "not configured" trains people to ignore red.**

The session precondition is a fixture, not a hook

```
export const test = base.extend<{ session: void }>({
  session: [async ({}, use) => {
    test.skip(!hasSession(), NO_SESSION_REASON);
    await use();
  }, { auto: true }],
});
```

The first version of this was a `test.beforeEach` in the shared helper module. A hook declared in an imported module attaches to whichever spec file happened to be loading at import time, so it applied to some files and not others depending on load order — the same suite reported 122 skips on one run and 24 failures on the next. A fixture is attached per test by the runner and cannot drift. Worth knowing before writing another shared helper.

Sign-in is captured, never scripted

`npm run e2e:login` opens a headed browser, waits up to ten minutes for a human to reach `/dashboard`, confirms `/authz/me` has answered, and saves `storageState` to `e2e/.auth/user.json` (gitignored — it is a live session token).

Nothing in this suite types a credential, creates an account, or automates a verification code. That is not squeamishness; it is that the alternatives are all worse. Provisioning a test user through the Clerk admin API mutates a shared identity provider from a test run. A hardcoded password could not work on this instance anyway. And bypassing the Clerk proxy with a test-only backdoor would mean the thing under test is no longer the thing that ships.

The cost is one manual step per session lifetime. That is the right trade for a console whose entire value is that it is hard to get into.

What each layer is for

Layer 1 — the session wall (33 tests, no session)

Every route asserted to bounce a signed-out visitor. This is the single highest-value unauthenticated check in the suite, because a regression exposes buyers, money and audit to anyone holding a URL. It also stays meaningful with no session at all, which is why it gates every commit.

Layer 2 — the API ingress (10 tests, no session)

The proxy exists so the Clerk JWT never enters the browser bundle. The tests assert the two properties that makes true: nothing is forwarded upstream without a session, and no token, JWT, secret key or backend origin appears in anything the client can read. Every request uses `maxRedirects: 0`, because following the redirect is exactly what hides the refusal — and that detail is how finding #1 in the report was found.

Layer 2b — the whole API surface (105 tests, no session)

`e2e/support/api-surface.ts` transcribes every endpoint `src/lib/api/hooks.ts` calls — 88 of them — and classifies each one:

KIND	MEANING	CALLED FOR REAL?
`read`	Safe against any environment	Yes
`write`	Creates or edits a record	Refusals only
`act`	Advances a state machine — money moves, ownership changes	Refusals only

That classification is the safety rail the rest of the suite is built on. A test can look up an endpoint and know whether calling it is a question or a decision.

Unauthenticated, all 88 are asserted to be refused — exhaustively, not by sample. This is the broadest security claim the suite makes and the only one that can be made completely without a session. Every request uses `maxRedirects: 0`; following the redirect lands on the sign-in page with a 200, and a suite that accepted that would report a wide-open API as secure. That exact mistake is how finding #1 surfaced.

Authenticated, 20 contract tests cover what the console reads directly and would break quietly without: the envelope, the omission rule checked at the payload (a `null` section collapses "not permitted" into "no data"), `pageSize` honoured, filters genuinely narrowing, masked scopes not returning unmasked emails, and no write acting on an empty body. It is not a re-test of the backend's business logic — that is the backend's suite to own.

Layer 3 — screens and flows (authenticated)

Every route is opened **cold, by URL**, not clicked into. Deep-linking is the path that breaks first, because data, permissions and URL parameters all arrive at once — and it is the real path, since operators arrive from bookmarks and pasted links.

Each route must reach one of exactly three legible states:

1. its own screen, identified by the `<h1>` it pushes to the topbar;
2. a denial that names the permission it would take;
3. an error state that says what went wrong.

Anything else — a blank frame, a skeleton that never resolves, an unhandled throw — fails. "It rendered *something*" is not a passing condition.

Layer 4 — permission gating, in both directions

This is the layer most suites get half-right. The assertions read `/authz/me` and compare it to what the UI offers, checking both directions:

- **Offered but refused.** A button that answers 403 is worse than no button; it teaches operators their tools are broken.
- **Held but hidden.** A capability the role holds and the UI never surfaces is a feature paid for and not delivered. Almost nobody tests this direction.

Two traps in this codebase get their own tests, because a naive capability check waves both through and the controller then refuses:

- **`configure` is not implied by `manage`.** `buildPaymentAccess` resolves `canConfigure` from a literal `payment-royalty:configure` key. Offering the gateway screen on a manage grant produces buttons the backend rejects.
- **Refunds need two grants.** The four refund routes are guarded by `order:refund:global`, but every handler first calls `buildPaymentAccess`, which throws 403 without a `payment-royalty` grant. `HITBOX_ORDER_MANAGER` holds the first and not the second — so the role named for refunds is refused on the workflow it was named for.

Because the assertions are about the *relationship* between grants and UI, they are correct for every role. Running the suite under two different roles is worth more than running it twice under one.

Layer 5 — the omission contract

The dashboard tests read the live payload, compute which sections are absent, and assert the UI treated absence as absence: the withheld panel names the right count, and no hidden figure is rendered as `0`. This is the one rule in the console where being confidently wrong costs the most, because the figure in question is money.

Layer 5b — the workflows, read-only and negative

Nine operator state machines: refund, payout run, release approval, order lifecycle, royalty reversal, role lifecycle, role assignment, dispute, minting.

They are testable without mutating anything because of how the console is built: the server returns `allowedTransitions` on each record, and the drawer's buttons are derived from it. So the assertions are about agreement, not about outcomes:

- the buttons **offered** equal the transitions **allowed**, checked in both directions — an offered-but-disallowed button 403s; an allowed-but-unoffered one strands the operator;
- an illegal transition asked of the server directly is refused — the refusal *is* the assertion, so nothing changes;
- preconditions are collected before submission: a rejection reason, an override reason, a confirmation on the two steps that cannot be undone;
- the permission traps hold — `reverse` needs an exact `payment-royalty:override:global`, not `manage`; payout actions need `manage:global` specifically;
- the ledger offers no edit affordance, because ledger rows are immutable.

Success paths are written and gated behind `E2E_ALLOW_WRITES=1`. They do not run without it, and the skip message says exactly what each one would do. `process` on a refund revokes ownership, quarantines the tag, sends money and reverses a royalty in one call; `execute` on a payout pays real people. Those are not test fixtures.

Layer 6 — resilience, by interception

`page.route()` on `/api/hitbox/**` synthesises 500, 403, connection-refused, 502, a 200 carrying HTML, and a slow response. `/authz/me` is deliberately passed through: breaking it stops the shell rendering, and the test becomes about the gate rather than the screen.

These are deterministic, touch no real data, and produce states a healthy backend never will — which is the point. An operator meets these on the worst day of the quarter.

Layer 7 — accessibility and layout

axe with **WCAG 2.1 A and AA only**. `best-practice` is excluded deliberately: it flags house-style opinions (heading order inside a card, landmarks around a data table), and mixing opinions into a gate teaches people to ignore the gate.

Contrast is scanned **separately** from everything else. There is one known systemic contrast defect, and inside the general scan it would mask every new violation behind an already-red test. Split, the general scan stays green and keeps doing its job.

Layout checks assert **no horizontal overflow** at four widths. A console that scrolls sideways on a phone is unusable in a warehouse, and the failure is invisible on a developer's monitor.

Known defects are annotated, not deleted

A test that documents a real defect is written to assert the **intended** behaviour and marked `test.fail()` with the reason:

```
test.fail(true, "Known: --text-3 on --surface-1 is 3.7:1, AA needs 4.5:1");
```

This does three things a skip or a deletion cannot:

- the suite stays green, so red keeps meaning "something changed";
- the defect stays executable, so it cannot rot into a stale comment;

- **when someone fixes it, the test passes unexpectedly and Playwright fails the run** — which is the prompt to delete the annotation.

Six such tests exist today. Each one is a finding in `E2E-TEST-REPORT.md`.

What this suite deliberately does not do

Each of these is a decision, not an oversight.

Write operations against the shared backend. No test creates a role, assigns a permission, posts an adjustment, approves a refund or publishes a drop. The backend is shared and these writes are not reversible from the UI — approving a refund moves money. The write path needs a seeded, disposable environment, and until one exists the honest position is a read-only suite plus this paragraph. What *is* covered is everything up to the commit point: the form renders, the button is correctly enabled or disabled, the validation refuses, the permission gate holds, and the server refuses the illegal step.

The success paths are nonetheless *written*, behind `E2E_ALLOW_WRITES=1`:

```
E2E_ALLOW_WRITES=1 HITBOX_API_URL=<seeded backend> npm run e2e:authed
```

Point that at a disposable environment and the workflow suite completes. Point it at production and it will do exactly what it says on the tin, which is why it is off by default and why every gated test's skip message names the consequence.

Account creation. No test signs up. The sign-up spec stops where a real identity would be submitted.

Visual regression. Screenshot diffing on a design this typography-heavy produces false positives on font rendering across machines. The layout assertions here are structural — overflow, in-viewport, target size — and hold across renderers. Worth revisiting with a pinned container image.

Unit-level permission logic. Already covered, and better, by `npm run verify:gating`: 49 assertions on parsing, scope ranking and nav derivation, with grant fixtures copied verbatim from the backend's `role-catalog.ts`. E2E asserts that the *UI* agrees with that logic; it does not re-test the logic.

Media upload. The two-step presigned flow is modelled but no upload form ships, and the backend's own presign calls have never run against a live bucket.

CSV export. `reports-dashboards:export:global` exists in the catalog with no consumer.

Running it

```
npm run e2e            # public suite, desktop + mobile — no session needed
npm run e2e:login      # capture a session (headed, interactive, once)
npm run e2e:authed     # the 210 authenticated tests
npm run e2e:all        # both
npm run e2e:ui         # watch mode
npm run e2e:report     # last HTML report
```

`playwright.config.ts` reuses an already-running dev server rather than starting a second one, so `npm run dev` in another terminal is fine.

Environment notes

Browsers. Playwright's bundled downloads (`chromium`, `chrome-headless-shell`, `ffmpeg`) fail behind some networks with `code=1`. The config therefore drives the **installed Chrome** by default and leaves video capture behind `E2E_VIDEO=1`. Traces are on for failures and carry a DOM snapshot per action, which beats a video for diagnosis anyway. In CI, run `npx playwright install chromium` and set `E2E_CHANNEL=chromium` to pin the version instead of inheriting whatever Chrome the agent has.

Never set `extraHTTPHeaders` globally. Playwright applies them to every request the page makes, including the cross-origin `<script>` fetch for clerk-js. A custom header makes that request non-simple, CORS preflights it, Clerk's CDN answers with a redirect, and Chrome refuses: *"Redirect is not allowed for a preflight request."* Clerk then never loads and the sign-in form silently does nothing — indistinguishable from a real auth outage. There is a comment in the config saying so; this is the second place, because it cost an hour once.

Where to take it next

Roughly in order of value per unit of effort.

1. **Run the authenticated suite under two roles** — one broad (System Admin), one narrow (Support or Order Manager). The gating assertions are written to be role-agnostic precisely so this is a matter of capturing a second session, and the narrow role is where the interesting refusals live.
2. **A seeded backend**, which unlocks the write path: role creation, assignment, refund workflow, adjustments. This is the largest remaining coverage gap and the one most worth closing.
3. **CI wiring.** `npm run e2e` needs only a dev server, so it can gate merges today. Authenticated runs go nightly against a long-lived session, with the capture step scheduled rather than ad-hoc.
4. **Fix finding #1 and delete two `test.fail` annotations.** The tests are already written; the fix makes them pass.
5. **Visual regression in a pinned container**, once the layout stabilises.